

MOVIE THEATRE WEBSITE SOFTWARE DESIGN SPECIFICATION

Version 1.0

Group 21
Nate Crivelli, Matthew Chung

Prepared for CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.

SOFTWARE DESIGN SPECIFICATION =====

SYSTEM OVERVIEW -----

The Movie Theatre Website is a web-based ticket purchasing and theatre management system for one theatre location. Customers can browse movies that are currently showing or coming soon, read movie details, select a showtime, choose available seats, select ticket types, pay through an outside payment service, and receive an order confirmation. Customers may purchase tickets as guests or create an account to view previous orders and manage basic profile information.

Authorized theatre administrators use a separate protected area to manage movies, theatres, showtimes, ticket types, prices, and orders. The system also communicates with a relational database, a third-party payment service, and an email service.

The design reflects the following important requirements from the SRS:

1. A seat cannot be sold to more than one customer for the same showtime.
2. A customer can temporarily reserve selected seats during checkout.
3. Temporary reservations expire if checkout is not completed.
4. An order is confirmed only after payment is approved.
5. Complete payment-card numbers and security codes are never stored.
6. Showtimes in the same theatre cannot overlap, including cleaning time.
7. Administrators can manage movie, theatre, showtime, price, and order data.
8. Registered customers can retrieve their order history.
9. Guests can retrieve an order by providing an order number and matching email.
10. The system supports one theatre location, English, and United States dollars in the first release.

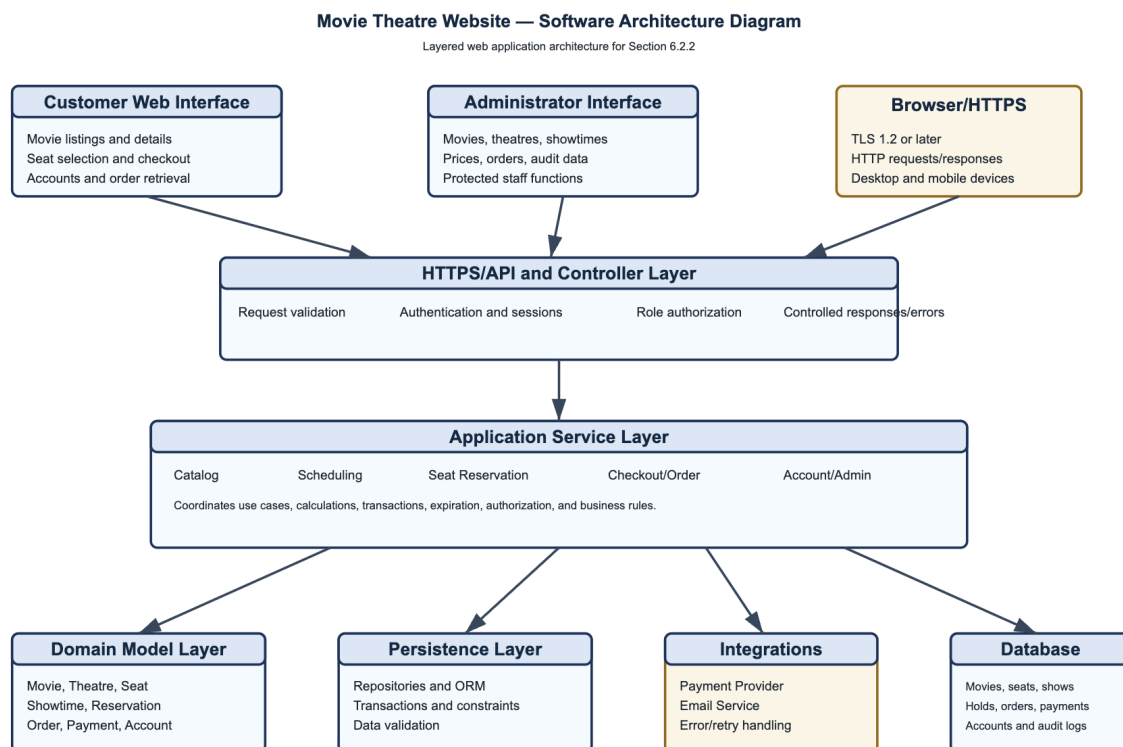
SOFTWARE ARCHITECTURE OVERVIEW

Architectural style

The system uses a layered web application architecture. The presentation layer contains the public customer website and the protected administration website. The application layer coordinates use cases and enforces business rules. The domain layer represents movies, theatres, seats, showtimes, reservations, orders, payments, and user accounts. The persistence layer stores and retrieves data from a relational database. An integration layer communicates with the outside payment and email services.

The browser communicates with the application server through HTTPS using TLS 1.2 or later. The browser must not communicate directly with the database, payment provider, or email service.

Software architecture diagram



Component descriptions

Customer Web Interface

Displays movie listings, movie details, showtimes, seating arrangements, checkout forms, confirmation pages, account pages, and order retrieval pages. It must clearly distinguish available, selected, reserved, sold, inaccessible, and accessible seats.

Administrator Interface

Provides authorized theatre employees with pages to add, edit, publish, unpublish, and archive movies; add, edit, cancel, and view showtimes; manage ticket types and prices; view orders; and review important administrative changes.

HTTPS/API and Controller Layer

Receives web requests, validates request data, establishes the current user or guest session, checks authorization, calls an application service, and returns an HTML or JSON response. This layer also converts service errors into clear, controlled messages.

Application Services

Coordinate complete use cases. Services enforce rules such as showtime conflict detection, temporary seat holds, reservation expiration, payment approval, order confirmation, and administrator permissions.

Domain Model

Contains the business objects and their validation behavior. The domain model does not depend on the browser or on a specific database implementation.

Repositories/Persistence

Provide interfaces for storing and retrieving domain objects. Database transactions and uniqueness constraints must prevent two confirmed orders from selling the same seat for one showtime.

External Integrations

The Payment Provider authorizes, captures, or reports payment results. The Email Service sends order-confirmation messages. The application stores only the outside payment reference and payment status, not complete card details.

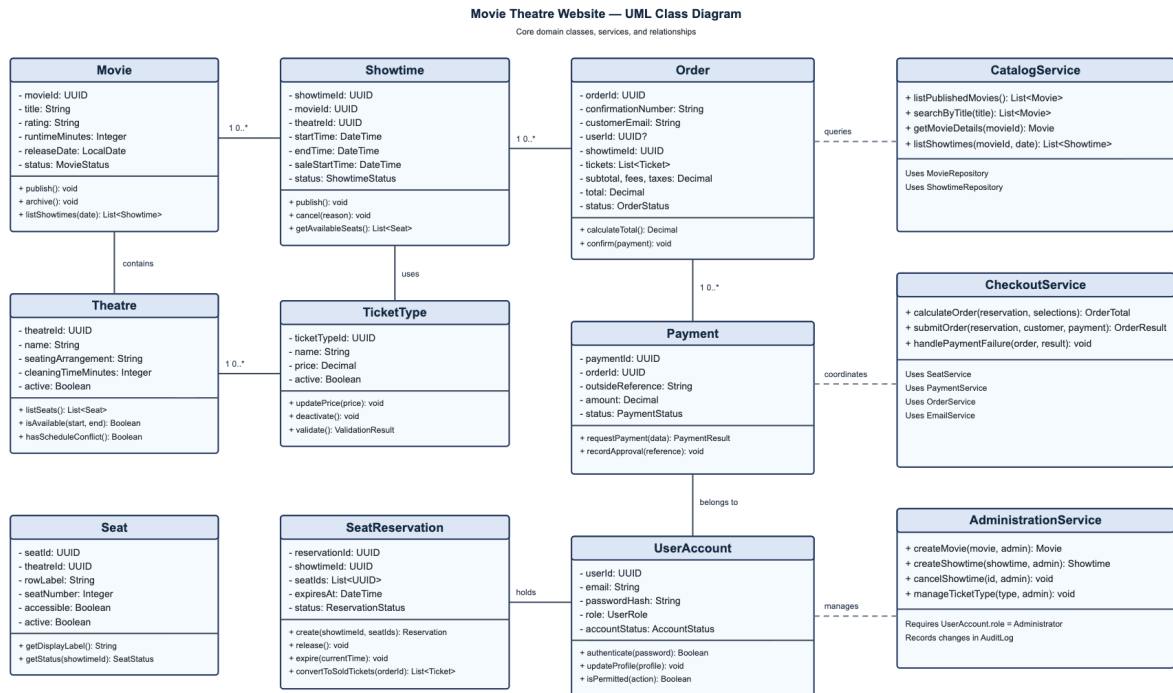
Relational Database

Stores movies, theatres, seats, showtimes, temporary reservations, orders, payments, user accounts, ticket prices, and administrator audit records. Each entity has a unique identifier, monetary values use decimal data types, and passwords are stored as secure hashes.

Architecture rules

1. The browser must use HTTPS and TLS 1.2 or later.
2. Controllers must call services instead of changing database records directly.
3. All seat availability checks must be repeated on the server when seats are held and again when an order is confirmed.
4. Seat holds, payment status, order status, and sold-seat updates must use transactional operations where necessary.
5. Only authorized administrator accounts may call administrative services.
6. Complete card numbers and security codes must never be written to the database, application logs, confirmation emails, or audit records.
7. External-service failures must result in controlled error handling and must not create a falsely confirmed order.

UML CLASS DIAGRAM



CLASS DESCRIPTIONS

=====

The following descriptions explain the purpose, important attributes, and operations of the major classes in the Movie Theatre Website. UUID values are unique identifiers. DateTime represents a date and time, LocalDate represents a calendar date, Decimal represents currency or other precise numeric values, and List<T> represents a collection of values of type T.

DOMAIN CLASSES

Movie

Purpose: Represents a movie that can be displayed in the customer catalogue and scheduled for theatre showings.

Attributes: movieId: UUID identifies the movie; title: String stores the movie name; description: String stores the synopsis; rating: String stores the content rating; runtimeMinutes: Integer stores the runtime; releaseDate: LocalDate stores the release date; posterUrl and trailerUrl: String store media links; status: MovieStatus identifies whether the movie is published, unpublished, or archived.

Operations: validate(): ValidationResult checks required fields and valid values; publish(), unpublish(), and archive() change the movie status; updateDetails(details: MovieDetails): void changes editable movie information;

listShowtimes(date: LocalDate): List<Showtime> returns showtimes for a date.

Theatre

Purpose: Represents the theatre location and its seating configuration.

Attributes: theatreId: UUID identifies the theatre; name: String stores its name; seatingArrangement: String describes the layout; cleaningTimeMinutes: Integer stores the cleaning time required between showings; active: Boolean indicates whether the theatre can be scheduled.

Operations: listSeats(): List<Seat> returns the theatre's seats;

isAvailable(start: DateTime, end: DateTime): Boolean checks whether the theatre can be used during a period; hasScheduleConflict(start: DateTime, end: DateTime): Boolean checks overlapping showtimes and cleaning time; updateDetails(...): void updates theatre information.

Seat

Purpose: Represents one physical seat in a theatre.

Attributes: seatId: UUID identifies the seat; theatreId: UUID identifies its theatre; rowLabel: String and seatNumber: Integer identify its location; seatType: SeatType identifies the category of seat; accessible: Boolean indicates accessibility; active: Boolean indicates whether it can be sold.

Operations: getDisplayLabel(): String returns a label such as A10;

canBeSold(): Boolean checks whether the seat is active and eligible for sale; getStatus(showtimeId: UUID): SeatStatus returns the seat's current status for a showtime.

Showtime

Purpose: Represents a scheduled showing of a movie in a theatre.

Attributes: showtimeId, movieId, theatreId, and priceScheduleId: UUID link the showtime to its records; startTime, endTime, saleStartTime, and saleEndTime: DateTime define the showing and sales period; status: ShowtimeStatus stores the current state.

Operations: validate(): ValidationResult checks time and relationship rules;

publish(): void makes the showing available for customers; cancel(reason: String): void cancels it and records the reason; hasConflict(other: Showtime, cleaningTimeMinutes: Integer): Boolean checks schedule overlap; getAvailableSeats(): List<Seat> returns seats that can be selected; getSeatMap(): SeatMap returns the complete seating display.

TicketType

Purpose: Defines a ticket category, such as adult, child, or senior.

Attributes: ticketTypeId: UUID identifies the type; name and description: String describe it; price: Decimal stores the base price; active: Boolean indicates whether customers may select it.

Operations: validate(): ValidationResult checks the name and price;

updatePrice(price: Decimal): void changes the price; deactivate(): void prevents the type from being used for new purchases.

PriceSchedule

Purpose: Stores pricing rules for a showtime.

Attributes: priceScheduleId and showtimeId: UUID identify the schedule and associated showtime; ticketTypes: List<TicketType> stores available ticket prices; taxRate and serviceFee: Decimal store checkout charges.

Operations: getPrice(ticketTypeId: UUID): Decimal returns a ticket price; calculateTax(subtotal: Decimal): Decimal calculates tax; calculateFee(subtotal: Decimal): Decimal calculates the service fee; calculateTotal(subtotal: Decimal, discount: Decimal): Decimal calculates the final amount.

SeatReservation

Purpose: Temporarily holds seats while a customer completes checkout.

Attributes: reservationId: UUID identifies the hold; showtimeId: UUID identifies the showing; seatIds: List<UUID> contains held seats; orderId: UUID? optionally links the hold to an order; createdAt and expiresAt: DateTime store the hold period; status: ReservationStatus stores its state.

Operations: create(showtimeId: UUID, seatIds: List<UUID>): SeatReservation creates a hold after availability is checked; release(): void releases seats; expire(currentTime: DateTime): void expires an old hold; convertToSoldTickets(orderId: UUID): List<Ticket> converts held seats into tickets after successful payment; isActive(currentTime: DateTime): Boolean checks whether the hold is still valid.

Order

Purpose: Represents a customer's ticket purchase from checkout through confirmation, cancellation, or refund.

Attributes: orderId: UUID identifies the order; confirmationNumber: String is shown to the customer; customerEmail: String stores the contact email; userId: UUID? optionally links a registered customer; showtimeId: UUID links the order to a showing; tickets: List<Ticket> stores purchased tickets; subtotal, fees, taxes, discount, and total: Decimal store monetary values; status: OrderStatus stores the order state; createdAt and confirmedAt: DateTime store timestamps.

Operations: calculateTotal(): Decimal calculates the total; confirm(payment: Payment): void confirms the order only after approved payment; cancel(reason: String): void cancels the order; getDetails(): OrderDetails returns customer-facing information; matchesRetrievalData(number: String, email: String): Boolean verifies guest retrieval information.

Ticket

Purpose: Represents one ticket and its assigned seat within an order.

Attributes: ticketId and orderId: UUID identify the ticket and order; seatId: UUID identifies the purchased seat; ticketTypeId: UUID identifies the ticket category; price: Decimal stores the amount charged; status: TicketStatus stores whether the ticket is sold, refunded, or otherwise unavailable.

Operations: validate(): Boolean checks required relationships; markSold(): void marks the ticket as sold after payment confirmation; refund(): void marks the ticket as refunded.

Payment

Purpose: Records the result of communicating with the outside payment provider without storing complete card information.

Attributes: paymentId and orderId: UUID identify the payment and order; outsidePaymentReference: String stores the provider reference; amount: Decimal stores the payment amount; status: PaymentStatus stores the result; authorizationDate: DateTime? stores approval time; failureReason: String? stores a failure explanation when applicable.

Operations: requestPayment(paymentData: PaymentData): PaymentResult starts a payment request; recordApproval(reference: String, date: DateTime): void records approval; recordFailure(reason: String): void records failure; checkUnknownResult(): PaymentResult checks an uncertain provider result.

UserAccount

Purpose: Represents a registered customer or authorized administrator.

Attributes: userId: UUID identifies the user; email: String is the login and contact address; passwordHash: String stores a secure password hash; role: UserRole identifies customer or administrator permissions; verificationStatus: VerificationStatus stores account verification; accountStatus: AccountStatus stores whether the account is active; createdAt: DateTime stores creation time.

Operations: authenticate(password: String): Boolean verifies login credentials; resetPassword(token: String, newPassword: String): Boolean changes a password after token validation; updateProfile(profile: ProfileData): void updates profile fields; isPermitted(action: AdminAction): Boolean checks authorization.

APPLICATION SERVICE CLASSES

CatalogService

Purpose: Provides read-only catalogue and showtime operations to the customer interface.

Operations: listPublishedMovies(status: MovieStatus): List<Movie> returns customer-visible movies; searchByTitle(title: String): List<Movie> searches

movie titles; getMovieDetails(movieId: UUID): Movie retrieves one movie;
listShowtimes(movieId: UUID, date: LocalDate): List<Showtime> returns valid
showtimes. Uses movie and showtime repositories.

SeatService

Purpose: Controls seat-map retrieval, temporary holds, releases, expiration,
and server-side availability checks.

Operations: getSeatMap(showtimeId: UUID): SeatMap returns seat statuses;
holdSeats(showtimeId: UUID, seatIds: List<UUID>): SeatReservation creates a
temporary hold; releaseReservation(reservationId: UUID): void releases a hold;
expireReservations(currentTime: DateTime): Integer expires old holds and
returns the number affected; verifyAvailable(showtimeId: UUID,
seatIds: List<UUID>): Boolean checks availability. Uses seat, showtime, and
reservation repositories.

CheckoutService

Purpose: Coordinates ticket selection, order creation, payment, and customer
notifications.

Operations: calculateOrder(reservationId: UUID, ticketSelections:
List<TicketSelection>): OrderTotal calculates charges; submitOrder(
reservationId: UUID, customerData: CustomerData, paymentData: PaymentData):
OrderResult creates and submits an order; handlePaymentFailure(orderId: UUID,
result: PaymentResult): void releases or updates the order after failure.
Uses SeatService, PaymentService, OrderService, and EmailService.

OrderService

Purpose: Retrieves, cancels, and manages customer orders.

Operations: getOrder(orderId: UUID): Order retrieves an order;
getCustomerOrders(userId: UUID): List<Order> returns a registered customer's
history; retrieveGuestOrder(number: String, email: String): Order retrieves a
guest order after matching both values; cancelOrder(orderId: UUID,
requester: UserAccount): Boolean checks permission and cancels an order.

PaymentService

Purpose: Provides an application-level interface to the external payment
provider.

Operations: authorize(paymentData: PaymentData, amount: Decimal):
PaymentResult requests authorization; capture(reference: String, amount:
Decimal): PaymentResult captures an authorized payment; refund(payment:
Payment): PaymentResult requests a refund.

EmailService

Purpose: Sends customer notifications without exposing sensitive payment data.

Operations: sendOrderConfirmation(order: Order): DeliveryResult sends a

confirmation; sendPaymentFailure(email: String, reason: String):
DeliveryResult reports a failed payment; sendCancellationNotice(order: Order):
DeliveryResult informs the customer of cancellation.

AdministrationService

Purpose: Provides protected operations for authorized theatre administrators.
Operations: createMovie(movie: Movie, administrator: UserAccount): Movie;
updateMovie(movie: Movie, administrator: UserAccount): Movie;
archiveMovie(movieId: UUID, administrator: UserAccount): void;
createShowtime(showtime: Showtime, administrator: UserAccount): Showtime;
cancelShowtime(showtimeId: UUID, administrator: UserAccount): void;
manageTicketType(type: TicketType, administrator: UserAccount): void; and
viewOrder(orderId: UUID, administrator: UserAccount): Order. Every operation
checks administrator permissions and records important changes in AuditLog.

SUPPORTING CLASSES AND INTERFACES

AuditLog

Purpose: Records important administrator actions for accountability.
Attributes: auditId, administratorId, and entityId: UUID identify the log and
affected records; action, entityType, and changeSummary: String describe the
change; createdAt: DateTime records when it occurred.
Operation: record(): void saves the audit record without storing sensitive
payment-card data.

PaymentProvider

Purpose: Defines the interface that any outside payment integration must
implement.
Operations: authorize(amount: Decimal, paymentData: PaymentData):
PaymentResult; capture(reference: String, amount: Decimal): PaymentResult;
refund(reference: String, amount: Decimal): PaymentResult; and
checkStatus(reference: String): PaymentResult.

Repository<T>

Purpose: Defines common persistence operations for domain entities while
keeping the domain layer independent of a particular database technology.
Operations: findById(id: UUID): T retrieves an entity; save(entity: T): T
creates and returns an entity; update(entity: T): T persists modifications.

DESIGN NOTES

Controllers should call application services rather than changing database

records directly. Seat holds, payment status, order status, and sold-ticket updates should use transactional operations. The system must check seat availability when seats are held and again when payment is confirmed. Complete payment-card numbers and security codes must not be stored in domain objects, database records, logs, or emails.

```
CLASS Movie {  
  - movieId: UUID  
  - title: String  
  - description: String  
  - rating: String  
  - runtimeMinutes: Integer  
  - releaseDate: LocalDate  
  - posterUrl: String  
  - trailerUrl: String  
  - status: MovieStatus  
  + validate(): ValidationResult  
  + publish(): void  
  + unpublish(): void  
  + archive(): void  
  + updateDetails(details: MovieDetails): void  
  + listShowtimes(date: LocalDate): List<Showtime>  
}
```

```
CLASS Theatre {  
  - theatreId: UUID  
  - name: String  
  - seatingArrangement: String  
  - cleaningTimeMinutes: Integer  
  - active: Boolean  
  + listSeats(): List<Seat>  
  + isAvailable(start: DateTime, end: DateTime): Boolean  
  + hasScheduleConflict(start: DateTime, end: DateTime): Boolean  
  + updateDetails(name: String, arrangement: String, cleaningTime: Integer): void  
}
```

```
CLASS Seat {  
  - seatId: UUID  
  - theatreId: UUID  
  - rowLabel: String  
  - seatNumber: Integer  
  - seatType: SeatType  
  - accessible: Boolean  
  - active: Boolean
```

```
+ getDisplayLabel(): String
+ canBeSold(): Boolean
+ getStatus(showtimeId: UUID): SeatStatus
}
```

```
CLASS Showtime {
- showtimeId: UUID
- movieId: UUID
- theatreId: UUID
- startTime: DateTime
- endTime: DateTime
- saleStartTime: DateTime
- saleEndTime: DateTime
- status: ShowtimeStatus
- priceScheduleId: UUID
+ validate(): ValidationResult
+ publish(): void
+ cancel(reason: String): void
+ hasConflict(other: Showtime, cleaningTimeMinutes: Integer): Boolean
+ getAvailableSeats(): List<Seat>
+ getSeatMap(): SeatMap
}
```

```
CLASS TicketType {
- ticketTypeId: UUID
- name: String
- description: String
- price: Decimal
- active: Boolean
+ validate(): ValidationResult
+ updatePrice(price: Decimal): void
+ deactivate(): void
}
```

```
CLASS PriceSchedule {
- priceScheduleId: UUID
- showtimeId: UUID
- ticketTypes: List<TicketType>
- taxRate: Decimal
- serviceFee: Decimal
+ getPrice(ticketTypeId: UUID): Decimal
+ calculateTax(subtotal: Decimal): Decimal
+ calculateFee(subtotal: Decimal): Decimal
+ calculateTotal(subtotal: Decimal, discount: Decimal): Decimal
}
```

}

```
CLASS SeatReservation {  
  - reservationId: UUID  
  - showtimeId: UUID  
  - seatIds: List<UUID>  
  - orderId: UUID?  
  - createdAt: DateTime  
  - expiresAt: DateTime  
  - status: ReservationStatus  
  + create(showtimeId: UUID, seatIds: List<UUID>): SeatReservation  
  + release(): void  
  + expire(currentTime: DateTime): void  
  + convertToSoldTickets(orderId: UUID): List<Ticket>  
  + isActive(currentTime: DateTime): Boolean  
}
```

```
CLASS Order {  
  - orderId: UUID  
  - confirmationNumber: String  
  - customerEmail: String  
  - userId: UUID?  
  - showtimeId: UUID  
  - tickets: List<Ticket>  
  - subtotal: Decimal  
  - fees: Decimal  
  - taxes: Decimal  
  - discount: Decimal  
  - total: Decimal  
  - status: OrderStatus  
  - createdAt: DateTime  
  - confirmedAt: DateTime?  
  + calculateTotal(): Decimal  
  + confirm(payment: Payment): void  
  + cancel(reason: String): void  
  + getDetails(): OrderDetails  
  + matchesRetrievalData(number: String, email: String): Boolean  
}
```

```
CLASS Ticket {  
  - ticketId: UUID  
  - orderId: UUID  
  - seatId: UUID  
  - ticketTypeId: UUID
```

```
- price: Decimal
- status: TicketStatus
+ validate(): Boolean
+ markSold(): void
+ refund(): void
}
```

```
CLASS Payment {
- paymentId: UUID
- orderId: UUID
- outsidePaymentReference: String
- amount: Decimal
- status: PaymentStatus
- authorizationDate: DateTime?
- failureReason: String?
+ requestPayment(paymentData: PaymentData): PaymentResult
+ recordApproval(reference: String, date: DateTime): void
+ recordFailure(reason: String): void
+ checkUnknownResult(): PaymentResult
}
```

```
CLASS UserAccount {
- userId: UUID
- email: String
- passwordHash: String
- role: UserRole
- verificationStatus: VerificationStatus
- accountStatus: AccountStatus
- createdAt: DateTime
+ authenticate(password: String): Boolean
+ resetPassword(token: String, newPassword: String): Boolean
+ updateProfile(profile: ProfileData): void
+ isPermitted(action: AdminAction): Boolean
}
```

```
CLASS CatalogService {
+ listPublishedMovies(status: MovieStatus): List<Movie>
+ searchByTitle(title: String): List<Movie>
+ getMovieDetails(movieId: UUID): Movie
+ listShowtimes(movieId: UUID, date: LocalDate): List<Showtime>
}
```

```
CLASS SeatService {
+ getSeatMap(showtimeId: UUID): SeatMap
}
```

```

+ holdSeats(showtimeId: UUID, seatIds: List<UUID>): SeatReservation
+ releaseReservation(reservationId: UUID): void
+ expireReservations(currentTime: DateTime): Integer
+ verifyAvailable(showtimeId: UUID, seatIds: List<UUID>): Boolean
}

```

```

CLASS CheckoutService {
+ calculateOrder(reservationId: UUID, ticketSelections: List<TicketSelection>): OrderTotal
+ submitOrder(reservationId: UUID, customerData: CustomerData, paymentData:
PaymentData): OrderResult
+ handlePaymentFailure(orderId: UUID, result: PaymentResult): void
}

```

```

CLASS OrderService {
+ getOrder(orderId: UUID): Order
+ getCustomerOrders(userId: UUID): List<Order>
+ retrieveGuestOrder(number: String, email: String): Order
+ cancelOrder(orderId: UUID, requester: UserAccount): Boolean
}

```

```

CLASS PaymentService {
+ authorize(paymentData: PaymentData, amount: Decimal): PaymentResult
+ capture(reference: String, amount: Decimal): PaymentResult
+ refund(payment: Payment): PaymentResult
}

```

```

CLASS EmailService {
+ sendOrderConfirmation(order: Order): DeliveryResult
+ sendPaymentFailure(email: String, reason: String): DeliveryResult
+ sendCancellationNotice(order: Order): DeliveryResult
}

```

```

CLASS AdministrationService {
+ createMovie(movie: Movie, administrator: UserAccount): Movie
+ updateMovie(movie: Movie, administrator: UserAccount): Movie
+ archiveMovie(movieId: UUID, administrator: UserAccount): void
+ createShowtime(showtime: Showtime, administrator: UserAccount): Showtime
+ cancelShowtime(showtimeId: UUID, administrator: UserAccount): void
+ manageTicketType(type: TicketType, administrator: UserAccount): void
+ viewOrder(orderId: UUID, administrator: UserAccount): Order
}

```

```

CLASS AuditLog {
- auditId: UUID

```

```

- administratorId: UUID
- action: String
- entityType: String
- entityId: UUID
- changeSummary: String
- createdAt: DateTime
+ record(): void
}

```

```

INTERFACE PaymentProvider {
+ authorize(amount: Decimal, paymentData: PaymentData): PaymentResult
+ capture(reference: String, amount: Decimal): PaymentResult
+ refund(reference: String, amount: Decimal): PaymentResult
+ checkStatus(reference: String): PaymentResult
}

```

```

INTERFACE Repository<T> {
+ findById(id: UUID): T
+ save(entity: T): T
+ update(entity: T): T
}

```

UML relationships and multiplicities

1. One Movie has zero or more Showtimes. Each Showtime belongs to exactly one Movie.
2. One Theatre has zero or more Seats. Each Seat belongs to exactly one Theatre.
3. One Theatre has zero or more Showtimes. Each Showtime belongs to exactly one Theatre.
4. One Showtime has one PriceSchedule and one or more possible Seats through its Theatre.
5. One Showtime has zero or more SeatReservations.
6. One SeatReservation contains one or more Seats and may be associated with one Order.
7. One Order contains one or more Tickets and may be associated with one registered UserAccount. Guest orders store an email without requiring a user account.
8. Each Ticket belongs to one Order, one Seat, and one TicketType.
9. One Order has zero or more Payment records, including possible retries or an unknown payment result.
10. CatalogService uses Movie and Showtime repositories.
11. SeatService uses Seat, Showtime, and SeatReservation repositories.
12. CheckoutService uses SeatService, PaymentService, OrderService, and

EmailService.

13. AdministrationService uses Movie, Theatre, Showtime, TicketType, Order, and AuditLog repositories.

14. PaymentService uses the PaymentProvider interface.

DEVELOPMENT PLAN AND TIMELINE

=====

The Movie Theatre Website will be developed incrementally. The team will first confirm the requirements and design, then implement the core customer and administrative features, and finally test the complete system. Work will be tracked through the group GitHub repository. Each team member will create and push commits for the work assigned to them.

Task Partitioning

1. Requirements and project documentation

- Review the original Software Requirements Specification.
- Keep the requirements, architecture, UML diagram, and implementation plan consistent as the project develops.
- Maintain the final project documentation and prepare the final submission.

2. Database and domain model

- Create the relational database tables for movies, theatres, seats, showtimes, reservations, orders, tickets, payments, users, and audit logs.
- Implement the domain objects, identifiers, status values, and validation rules described in the class specifications.

3. Customer website

- Implement movie listings, movie details, showtime selection, seat-map display, ticket selection, checkout, order confirmation, and guest order retrieval.
- Clearly display available, selected, temporarily reserved, sold, and inaccessible seats.

4. Administrator website

- Implement administrator authentication and authorization.
- Implement movie, theatre, showtime, ticket-type, price, and order management functions.
- Record important administrative actions in the audit log.

5. Reservation, checkout, and payment processing
 - Implement temporary seat holds and expiration handling.
 - Prevent two customers from purchasing the same seat for the same showtime.
 - Implement order totals, taxes, service fees, payment approval or failure, and order confirmation.
 - Ensure that complete payment-card numbers and security codes are never stored.
6. Testing and quality assurance
 - Test normal, invalid, and failure conditions for customer and administrator workflows.
 - Test showtime conflicts, seat-hold expiration, duplicate-seat prevention, payment failures, guest order retrieval, and administrator permissions.
 - Correct defects and verify that the final implementation agrees with the requirements and design specification.

Team Member Responsibilities

Nate Crivelli

- Lead the application architecture and backend/domain implementation.
- Implement or coordinate the database, repositories, domain classes, reservation logic, checkout logic, and payment-service interfaces.
- Review the security rules and verify that transactional seat and order operations are handled correctly.

Matthew Chung

- Lead project documentation and customer-facing functionality.
- Maintain the requirements/design documentation, UML updates, development plan, and final report sections.
- Implement or coordinate the customer movie listings, movie details, showtime selection, seat-map display, checkout pages, and order retrieval.
- Assist with integration testing, proofreading, defect tracking, and final submission preparation.

Both team members

- Review each other's work before merging changes.
- Use descriptive GitHub commit messages and push at least one commit each.
- Participate in integration testing and approve the final project version.

Development Timeline

Phase 1 - Requirements and design review

Activities: Review the SRS, confirm the architecture, verify the UML class diagram, and finalize the development plan.

Deliverable: Approved requirements and software design specification.

Phase 2 - Database and application foundation

Activities: Create the database schema, repositories, domain classes, authentication foundation, and project configuration.

Deliverable: Working data and application foundation.

Phase 3 - Customer functionality

Activities: Implement movie browsing, showtime selection, seat maps, ticket selection, checkout forms, guest retrieval, and customer order history.

Deliverable: Customer purchase workflow.

Phase 4 - Administration and integrations

Activities: Implement administrator functions, reservation expiration, payment-provider integration, email notifications, and audit logging.

Deliverable: Working administrative and external-service features.

Phase 5 - Integration and system testing

Activities: Connect all layers, test complete workflows, test security and concurrency rules, and correct defects.

Deliverable: Tested integrated system.

Phase 6 - Final review and submission

Activities: Review the implementation against the SRS, update diagrams and documentation, verify that both members have GitHub commits, and submit the final repository link.

Deliverable: Final project report and repository submission.